

GAmeetsLLM

December 21, 2025

1 Replicating ‘Geometric Algebra Meets Large Language Models’

1.1 Objective

This notebook replicates the methodology presented in the paper “**Geometric Algebra Meets Large Language Models: Instruction-Based Transformations of Separate Meshes in 3D, Interactive and Controllable Scenes**” (arXiv:2408.02275).

The paper introduces **Shenlong**, a system that leverages Large Language Models (LLMs) to generate Geometric Algebra (GA) operations for precise 3D scene editing. Instead of relying on coordinate-heavy matrices or complex scripts, Shenlong uses GA as a “communicative mediator” or neurosymbolic bridge.

1.1.1 Key Concepts from the Paper:

1. **Neurosymbolic AI**: Using LLMs to generate symbolic mathematical expressions (GA) rather than direct numerical values.
2. **Geometric Algebra (GA)**: A robust mathematical framework for spatial transformations. While the paper utilizes **Conformal Geometric Algebra (CGA)**, this notebook implements the concepts using **Projective Geometric Algebra (PGA)** (specifically $\mathbb{R}^{3,0,1}$), which is highly efficient for Euclidean rigid body mechanics and conceptually similar for translations and rotations.
3. **Instruction-Based Editing**: Translating natural language (e.g., “put the cube on top of the other one”) into mathematical operations (Motors).

1.2 Dependencies

We use `kingdon` for the Algebra (PGA/CGA) and `torch` for tensor operations.

```
[32]: %pip install kingdon torch numpy
```

```
Requirement already satisfied: kingdon in  
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages  
(1.4.0)
```

```
Requirement already satisfied: torch in  
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages  
(2.8.0)
```

```
Requirement already satisfied: numpy in  
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages  
(1.24.4)
```

Requirement already satisfied: sympy in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from kingdon) (1.13.3)

Requirement already satisfied: anywidget in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from kingdon) (0.9.18)

Requirement already satisfied: filelock in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from torch) (3.13.1)

Requirement already satisfied: typing-extensions>=4.10.0 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from torch) (4.14.1)

Requirement already satisfied: networkx in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from torch) (3.3)

Requirement already satisfied: jinja2 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from torch) (3.1.6)

Requirement already satisfied: fsspec in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from torch) (2024.6.1)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from sympy->kingdon) (1.3.0)

Requirement already satisfied: ipywidgets>=7.6.0 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from anywidget->kingdon) (8.1.7)

Requirement already satisfied: psygnal>=0.8.1 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from anywidget->kingdon) (0.15.0)

Requirement already satisfied: comm>=0.1.3 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from ipywidgets>=7.6.0->anywidget->kingdon) (0.2.3)

Requirement already satisfied: ipython>=6.1.0 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from ipywidgets>=7.6.0->anywidget->kingdon) (8.37.0)

Requirement already satisfied: traitlets>=4.3.1 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from ipywidgets>=7.6.0->anywidget->kingdon) (5.14.3)

Requirement already satisfied: widgetsnbextension~=4.0.14 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from ipywidgets>=7.6.0->anywidget->kingdon) (4.0.14)

Requirement already satisfied: jupyterlab_widgets~=3.0.15 in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from ipywidgets>=7.6.0->anywidget->kingdon) (3.0.15)

Requirement already satisfied: decorator in
/Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
(from ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (5.2.1)

Requirement already satisfied: exceptiongroup in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (1.3.0)

Requirement already satisfied: jedi>=0.16 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (0.19.2)

Requirement already satisfied: matplotlib-inline in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (0.1.7)

Requirement already satisfied: pexpect>4.3 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (4.9.0)

Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (3.0.51)

Requirement already satisfied: pygments>=2.4.0 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (2.19.1)

Requirement already satisfied: stack_data in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (0.6.3)

Requirement already satisfied: wcwidth in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from prompt_toolkit<3.1.0,>=3.0.41->ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (0.2.13)

Requirement already satisfied: parso<0.9.0,>=0.8.4 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from jedi>=0.16->ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (0.8.4)

Requirement already satisfied: ptyprocess>=0.5 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from pexpect>4.3->ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon)
 (0.7.0)

Requirement already satisfied: MarkupSafe>=2.0 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from jinja2->torch) (3.0.2)

Requirement already satisfied: executing>=1.2.0 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from stack_data->ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (2.2.0)

Requirement already satisfied: asttokens>=2.1.0 in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from stack_data->ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (3.0.0)

Requirement already satisfied: pure-eval in
 /Users/papagian/opt/anaconda3/envs/Elements310/lib/python3.10/site-packages
 (from stack_data->ipython>=6.1.0->ipywidgets>=7.6.0->anywidget->kingdon) (0.2.3)

[notice] A new release of pip is
 available: 25.2 -> 25.3

[notice] To update, run:

```
pip install --upgrade pip
```

Note: you may need to restart the kernel to use updated packages.

```
[33]: import torch
import numpy as np
import json
from kingdom import Algebra

# =====
# 1. GEOMETRIC ALGEBRA SETUP (PGA 3D)
# =====
# The paper "Geometric Algebra Meets Large Language Models" advocates for using
↪GA
# as a symbolic language for LLMs.
# While the paper uses Conformal Geometric Algebra (CGA), we use Projective
↪Geometric Algebra (PGA)
# (R3,0,1) here. PGA is often preferred for rigid body mechanics in computer
↪graphics
# due to its flat, Euclidean nature and efficiency.
#
# Algebra Signature: (3, 0, 1) -> 3 positive squared basis vectors (e1, e2,
↪e3),
# 0 negative, 1 zero squared (e0).
alg = Algebra(3, 0, 1)

# Extract basis blades.
# e1, e2, e3 correspond to the X, Y, Z axes.
# e0 is the 'ideal' plane (or plane at infinity), used for translations.
e1, e2, e3, e0 = alg.blades.e1, alg.blades.e2, alg.blades.e3, alg.blades.e0

def T(vector):
    """
    Creates a Translator Motor from a vector.

    Paper Context:
    In the paper (CGA), a translation is  $T = 1 - 0.5 * t * e_{inf}$ .
    In PGA, the formula is structurally identical:  $T = 1 - 0.5 * t * e_0$ .

    Args:
        vector: A GA vector (e.g.,  $x*e1 + y*e2 + z*e3$ ) representing the
↪displacement.

    Returns:
        A multivector representing the translation motor.
    """
    # We wedge with e0 (boundary) to move to the ideal plane.
```

```

    # This creates a bivector term (e.g.,  $x e_1 e_0$ ) which generates the
    ↪ translation.
    return 1 - 0.5 * (vector * e0)

def R(theta, u, v):
    """
    Creates a Rotor representing rotation by theta in plane  $u \wedge v$ .

    Paper Context:
    Rotors are the standard way to represent rotations in GA, superior to Euler
    ↪ angles
    or matrices for interpolation and composition.

    Formula:  $R = \cos(\theta/2) - \sin(\theta/2) * B$ 
    where B is the normalized bivector of the rotation plane.

    Args:
        theta: Rotation angle in radians.
        u, v: Vectors defining the plane of rotation (e.g.,  $e_1, e_2$  for  $XY$ 
    ↪ plane).

    Returns:
        A multivector representing the rotation rotor.
    """
    bivector = u ^ v
    # Kingdom allows function application on multivectors
    # The minus sign is standard for the exponential map in this signature
    ↪ context
    return alg.math.cos(theta / 2) - alg.math.sin(theta / 2) * bivector

# =====
# 2. ELEMENTS FRAMEWORK LOGIC (ROBUST)
# =====
# We strictly model the Mesh logic used in Elements for scene graph updates.
# This class represents a "Separate Mesh" as discussed in the paper's
    ↪ methodology.
class GANode:
    def __init__(self, name, vertices_np):
        self.name = name
        self.vertices = vertices_np # (N, 3) numpy array
        self.update_bounds()

    def update_bounds(self):
        """
        Compute Axis Aligned Bounding Box (AABB) in GA terms.

        Paper Context:

```

The paper emphasizes using bounding boxes (min/max points) to allow the LLM to reason about spatial relationships (e.g., "on top of", "next to") without processing the full mesh geometry.

```

"""
self.min_np = np.min(self.vertices, axis=0)
self.max_np = np.max(self.vertices, axis=0)

# Convert bounds to GA vectors for the LLM template logic.
# These attributes (X1.min, X1.max) are exposed to the LLM context.
# Use alg.blades to ensure we use the correct PGA basis even if globals
change
e1, e2, e3 = alg.blades.e1, alg.blades.e2, alg.blades.e3
self.min = self.min_np[0]*e1 + self.min_np[1]*e2 + self.min_np[2]*e3
self.max = self.max_np[0]*e1 + self.max_np[1]*e2 + self.max_np[2]*e3

def apply_motor(self, motor):
    """
    Apply a PGA motor to the vertices.

    Mechanism:
    To transform a point P by a motor M, we use the "sandwich product":
     $P' = M * P * \sim M$ 
    where  $\sim M$  is the reverse of M.

    In PGA (R3,0,1), points are typically represented as trivectors (dual
    to planes).

    Standard embedding:  $P = e123 + x*e023 - y*e013 + z*e012$ 
    """
    # Blades for point construction/extraction (Dual PGA)
    e023 = alg.blades.e023 # Dual to e1 (x)
    e013 = alg.blades.e013 # Dual to e2 (y)
    e012 = alg.blades.e012 # Dual to e3 (z)
    e123 = alg.blades.e123 # Dual to e0 (origin/w)

    new_verts = []
    for v in self.vertices:
        # 1. Embed Euclidean point into PGA Trivector
        #  $P = e123$  (origin) +  $x*e023$  (x-dir) -  $y*e013$  (y-dir) +  $z*e012$ 
        (z-dir)
        # Note: The sign on y depends on the orientation of the basis e013
        vs e031.
        point = e123 + v[0]*e023 - v[1]*e013 + v[2]*e012

        # 2. Apply Transformation (Sandwich Product)

```

```

        # This is the core operation applied to every vertex.
        transformed = motor * point * ~motor

        # 3. Project back to Euclidean (normalization)
        # The scalar coefficient of e123 acts as the homogeneous coordinate
        ↪ 'w'.
        w = float(transformed.e123)
        if abs(w) < 1e-6:
            w = 1.0 # Avoid division by zero for ideal points (shouldn't
        ↪ happen for rigid motions)

        # Extract coordinates from the dual components
        tx = float(transformed.e023) / w
        ty = -float(transformed.e013) / w
        tz = float(transformed.e012) / w

        new_verts.append([tx, ty, tz])

    self.vertices = np.array(new_verts)
    self.update_bounds()

    def __repr__(self):
        return f"<{self.name}: Bounds [{self.min_np} to {self.max_np}]>"

# =====
# 3. SCENE SETUP
# =====
# Initialize a simple cube centered at origin
# This corresponds to the "Initial State" in the paper's examples.
cube_verts = np.array([
    [-1, -1, -1], [1, -1, -1], [1, 1, -1], [-1, 1, -1],
    [-1, -1, 1], [1, -1, 1], [1, 1, 1], [-1, 1, 1]
])

scene_objects = {
    'X1': GANode('X1', cube_verts.copy()),
    'X2': GANode('X2', cube_verts.copy() + np.array([5, 0, 0])) # Offset box
}

print("--- Initial State ---")
for obj in scene_objects.values():
    print(obj)

```

```

--- Initial State ---
<X1: Bounds [[-1 -1 -1] to [1 1 1]]>
<X2: Bounds [[ 4 -1 -1] to [6 1 1]]>

```

[]:

```
[34]: # =====
# 4. LLM INSTRUCTION SIMULATION
# =====
# The paper describes "Shenlong", a system where an LLM generates JSON
#   containing GA operations.
#
# Methodology from Paper:
# 1. User Query: "Place X2 on top of X1"
# 2. Template Matching: The LLM is prompted with the object names (X1, X2) and
#   GA primitives.
# 3. Symbolic Reasoning: The LLM generates a mathematical expression using
#   object properties (X1.max, X2.min).
# 4. Output: A JSON object mapping object IDs to GA transformation strings.

def verify_and_get_instruction(instruction_type):
    """
    Simulates the output of the LLM (GPT-4) as described in the paper.
    """
    if instruction_type == "translate_left":
        # Simple Query: Move X1 left by 1 unit.
        return json.dumps({
            "X1": "T(-1 * e1)"
        })
    elif instruction_type == "rotate_z":
        # Simple Query: Rotate 90 deg around Z (plane e1^e2).
        return json.dumps({
            "X1": "R(np.pi/2, e1, e2)"
        })
    elif instruction_type == "relative_stack":
        # Compositional/Fuzzy Query: "Place X2 on top of X1"
        #
        # Paper Logic:
        # The LLM must calculate the translation vector required to align the
        # surfaces.
        # Target Position = X1.top_surface
        # Current Position = X2.bottom_surface
        # Displacement = Target - Current
        #
        # GA Implementation:
        # We use the Inner Product (/) to extract scalar coordinates from the
        # vectors.
        # (X1.max / e2) extracts the Y-component of the max bound of X1.

        expression = (
            "T( "
```

```

        "((X1.max | e1) - (X2.min | e1)) * e1 + " # Align X (simplified
↪corner align)
        "((X1.max | e2) - (X2.min | e2)) * e2 + " # Stack Y (X1 max Y - X2
↪min Y)
        "((X1.max | e3) - (X2.min | e3)) * e3 " # Align Z
        ")"
    )
    return json.dumps({
        "X2": expression
    })

# Select a test instruction
# We simulate the "relative_stack" command, which corresponds to the paper's
↪example
# of placing a bottle on a table.
llm_output_json = verify_and_get_instruction("relative_stack")
print(f"\n--- Synthetic LLM Output ---\n{llm_output_json}")

```

--- Synthetic LLM Output ---

```
{
  "X2": "T( ((X1.max | e1) - (X2.min | e1)) * e1 + ((X1.max | e2) - (X2.min | e2)) * e2 + ((X1.max | e3) - (X2.min | e3)) * e3 )"
}
```

[35]:

```

# =====
# 5. EXECUTION ENGINE (The verification)
# =====
# This section corresponds to the "Shenlong" system backend.
# It parses the JSON, evaluates the GA expressions, and applies the
↪transformations.

def execute_transformations(json_data, scene_d, algebra_ctx):
    """
    Parses and executes the GA transformations generated by the LLM.

    Args:
        json_data: JSON string containing {object_id: ga_expression_string}
        scene_d: Dictionary of scene objects (GANode)
        algebra_ctx: The Kingdon Algebra instance
    """
    ops = json.loads(json_data)

    # Context for eval(): Must include GA basis and helper functions
    # This allows the string "T(e1)" to be evaluated as a function call T with
    ↪argument e1.
    eval_ctx = {
        'e1': algebra_ctx.blades.e1, 'e2': algebra_ctx.blades.e2, 'e3':
    ↪algebra_ctx.blades.e3,

```

```

    'np': np,
    'T': T, # Translator function
    'R': R  # Rotor function
}
# Add dynamic objects to context so expressions like "X1.max" work.
eval_ctx.update(scene_d)

for target_name, op_str in ops.items():
    if target_name not in scene_d:
        print(f"Error: {target_name} not found.")
        continue

    print(f"Executing on {target_name}: {op_str}")

    # 1. Semantic Parsing / Neurosymbolic Execution
    # We use Python's eval() to interpret the symbolic math string.
    # In a production system, a safer parser would be used, but this
    ↪ demonstrates
    # the power of the symbolic representation.
    try:
        motor = eval(op_str, {"__builtins__": {}}, eval_ctx)
    except Exception as e:
        print(f"Failed to evaluate GA expression: {e}")
        continue

    # 2. Physics Update: Apply motor to mesh
    # The motor (multivector) is applied to all vertices of the mesh.
    scene_d[target_name].apply_motor(motor)

# Run execution
execute_transformations(llm_output_json, scene_objects, alg)

print("\n--- Final State ---")
for obj in scene_objects.values():
    print(obj)

# =====
# 6. AUTOMATED VERIFICATION
# =====
# We stacked X2 on X1.
# X1 max Y is 1.0. X2 was originally at Y [-1, 1].
# It should move up by 2.0 units so X2.min Y approx equals X1.max Y (1.0)

x1_max_y = scene_objects['X1'].max_np[1]
x2_min_y = scene_objects['X2'].min_np[1]

```

```

assert np.isclose(x1_max_y, x2_min_y, atol=1e-5), f"Stacking check failed! X1_
↳Top: {x1_max_y}, X2 Bottom: {x2_min_y}"
print("\n[SUCCESS] Verification Passed: Object X2 is correctly stacked on X1_
↳based on GA calculation.")

```

Executing on X2: $T((X1_{max} | e1) - (X2_{min} | e1)) * e1 + ((X1_{max} | e2) - (X2_{min} | e2)) * e2 + ((X1_{max} | e3) - (X2_{min} | e3)) * e3$)

--- Final State ---

<X1: Bounds $[-1 \ -1 \ -1]$ to $[1 \ 1 \ 1]$ >

<X2: Bounds $[1. \ 1. \ 1.]$ to $[3. \ 3. \ 3.]$ >

[SUCCESS] Verification Passed: Object X2 is correctly stacked on X1 based on GA calculation.

2 Tutorial: Conformal Geometric Algebra (CGA) for Scene Editing

This tutorial provides a beginner-friendly introduction to **Conformal Geometric Algebra (CGA)**, the specific framework used in the paper “Geometric Algebra Meets Large Language Models”.

2.1 1. What is Conformal Geometric Algebra (CGA)?

While the previous sections of this notebook used **PGA** (Projective Geometric Algebra) for simplicity in rigid body mechanics, the paper utilizes **CGA** (Conformal Geometric Algebra). CGA is a 5-dimensional algebra that embeds 3D Euclidean space.

2.1.1 Why CGA?

CGA is even more powerful than PGA because it unifies: * **Points, Lines, Planes** * **Circles, Spheres** (Round objects are native elements!) * **Translations, Rotations** (Rigid Body Motions) * **Dilations** (Uniform Scaling) - *This is the key addition over PGA.*

In CGA, a “point” is a null vector, and transformations are “rotors” (versors) that preserve the angle (conformal structure).

2.2 2. The Basics of CGA ($\mathbb{R}^{4,1}$)

CGA uses a 5D vector space with signature (4, 1) (4 positive, 1 negative basis vector). * e_1, e_2, e_3 : Standard Euclidean basis. * e_+, e_- : Two extra dimensions.

We typically work with a **Null Basis** constructed from the extra dimensions: * e_o (origin): Represents the point at the origin. * e_∞ (infinity): Represents the point at infinity.

2.2.1 Key Operations

The operations (Geometric Product, Wedge Product) remain the same as in other GAs, but their geometric meaning expands.

2.3 3. Representing Objects in CGA

2.3.1 Points

A 3D point $\mathbf{x} = (x, y, z)$ is mapped to a vector P in 5D CGA:

$$P = \mathbf{x} + \frac{1}{2}\mathbf{x}^2\mathbf{e}_\infty + \mathbf{e}_o$$

This is a non-linear embedding (stereographic projection).

2.3.2 Motors (Transformations)

Transformations are represented as **Versors** (Rotors).

Translation To move by a vector $\mathbf{t} = (x, y, z)$:

$$T = 1 - \frac{1}{2}\mathbf{t}\mathbf{e}_\infty$$

(Notice how similar this is to PGA, but using $\mathbf{e}_\infty$ instead of $\mathbf{e}_0$)

Rotation To rotate by angle θ in a plane B (e.g., $\mathbf{e}_1 \wedge \mathbf{e}_2$):

$$R = \cos\left(\frac{\theta}{2}\right) - \sin\left(\frac{\theta}{2}\right)B$$

Dilation (Scaling) To scale uniformly by a factor s :

$$D = \exp\left(-\frac{\ln(s)}{2}E\right)$$

where $E = \mathbf{e}_o \wedge \mathbf{e}_\infty$ is the “flat point” at the origin.

Composition

$$M = TRD$$

You can compose Translation, Rotation, and Dilation simply by multiplying them.

2.4 4. The “Shenlong” Approach with CGA

The paper demonstrates that LLMs can generate these CGA operations effectively. For example, to “Scale the object by 2 and move it right”: 1. LLM generates D for scale 2. 2. LLM generates T for translation. 3. LLM combines them: $M = T * D$.

2.5 5. Try it yourself!

In the cells below, we will instantiate a CGA algebra and try these operations.

```
[36]: # Tutorial Playground: Conformal Geometric Algebra (CGA)
      from kingdom import Algebra
      import numpy as np
```

```

# 1. Setup CGA ( $R_4, 1$ )
# We create a new algebra instance for this tutorial section
alg_cga = Algebra(4, 1)
# Basis:  $e_1, e_2, e_3, e_4 (+), e_5 (-)$ 
e1, e2, e3, e4, e5 = alg_cga.blades.e1, alg_cga.blades.e2, alg_cga.blades.e3,
    ↪ alg_cga.blades.e4, alg_cga.blades.e5

# Define Null Basis ( $n_i = \text{infinity}, n_o = \text{origin}$ )
# Standard convention:  $n_i = e_4 + e_5, n_o = 0.5 * (e_5 - e_4)$ 
ni = e4 + e5
no = 0.5 * (e5 - e4)

def cga_point(x, y, z):
    """Embeds a 3D point into CGA."""
    v = x*e1 + y*e2 + z*e3
    #  $P = x + 0.5 * x^2 * n_i + n_o$ 
    return v + 0.5 * (v|v) * ni + no

def cga_translation(x, y, z):
    """Creates a translation rotor."""
    t = x*e1 + y*e2 + z*e3
    #  $T = 1 - 0.5 * t * n_i$ 
    return 1 - 0.5 * t * ni

def cga_rotation(angle_rad, plane_bivector):
    """Creates a rotation rotor."""
    # Using np functions for scalars
    return np.cos(angle_rad/2) - np.sin(angle_rad/2) * plane_bivector

def cga_dilation(scale):
    """Creates a dilation rotor."""
    #  $D = \exp(-\ln(s)/2 * (e_{inf} \wedge e_o))$ 
    # My  $E = n_o \wedge n_i = -(e_{inf} \wedge e_o)$ 
    # So  $D = \exp(\ln(s)/2 * E)$ 
    E = no ^ ni
    gamma = np.log(scale) / 2
    return np.cosh(gamma) + np.sinh(gamma) * E

# 2. Create a Point
p_orig = cga_point(1, 0, 0) # Point at (1, 0, 0)
print(f"Original Point: {p_orig}")

# 3. Create Transformations
# Translate by (2, 0, 0) -> Should end up at (3, 0, 0)
T = cga_translation(2, 0, 0)

# Rotate 90 deg around Z ( $e_1 \wedge e_2$  plane) -> (3, 0, 0) becomes (0, 3, 0)

```

```

R = cga_rotation(np.pi/2, e1^e2)

# Scale by 2 -> (0, 3, 0) becomes (0, 6, 0)
D = cga_dilation(2.0)

# Compose:  $M = D * R * T$  (Order matters! Applied right-to-left usually, or
↳ left-to-right depending on convention)
# Sandwich product:  $M * P * \sim M$ 
# If we want  $T$  then  $R$  then  $D$ :
#  $P' = D * (R * (T * P * \sim T) * \sim R) * \sim D$ 
#  $= (D * R * T) * P * \sim (D * R * T)$ 
M = D * R * T
print(f"Total Motor: {M}")

# 4. Apply Transformation
p_new = M * p_orig * ~M
print(f"Transformed Point (CGA): {p_new}")

# 5. Extract Coordinates
# To get back  $x, y, z$  from  $P = x + 0.5x^2 ni + no$ :
# We can project.
# Or use inner product with basis vectors?
#  $P \cdot ni = (x + \dots + no) \cdot ni = no \cdot ni = -1$ 
#  $P \cdot e1 = x$  (roughly, if normalized)
# Let's normalize first (ensure coefficient of  $no$  is 1)
# We extract the scalar part of the inner product
# Kingdom stores values in .values(). For a scalar  $MV$ , it might be a list or
↳ array.
w_mv = (p_new | ni)
# Try to get the scalar value safely
try:
    w = w_mv.values()[0]
except:
    w = float(w_mv)

print(f"Weight w: {w}")

# Note:  $w$  should be  $-1$  for normalized point.
if abs(w) > 1e-6:
    p_norm = p_new / (-w)
else:
    p_norm = p_new

# Extract coordinates
x_new = (p_norm | e1)
y_new = (p_norm | e2)
z_new = (p_norm | e3)

```

```

# Handle scalar extraction for coordinates too
def get_scalar(mv):
    try:
        return mv.values()[0]
    except:
        return float(mv)

x_new = get_scalar(x_new)
y_new = get_scalar(y_new)
z_new = get_scalar(z_new)

print(f"New Coordinates: ({x_new:.2f}, {y_new:.2f}, {z_new:.2f})")
print("Expected: (0.00, 6.00, 0.00)")

```

```

Original Point: 1    + 1.0
Total Motor: 0.75 + -0.75    + -1.0    + -1.0    + -1.0    + -1.0    +
-0.25    + 0.25
Transformed Point (CGA): 7.77e-16    + 3.0    + 8.75    + 9.25    + 2.22e-16
    + 2.22e-16    + 3.33e-16    + -3.33e-16
Weight w: -0.5
New Coordinates: (0.00, 6.00, 0.00)
Expected: (0.00, 6.00, 0.00)

```

3 7. Visualization in Elements Framework

This section demonstrates how to visualize the resulting scene using the **Elements** framework (pyECSS + pyGLV). We will convert the **GANode** objects (which contain the transformed vertices) into **Elements Entities** with **RenderMesh** components and render them in a 3D window.

Note: Running the cell below will open a separate OpenGL window. You may need to close the window to continue interacting with the notebook kernel.

```

[ ]: import sys
import os
# Add the project root to the path so we can import Elements
# We need to add the 'src' folder to sys.path
# Path: neuralCG -> notebooks -> pyEEL -> Elements (Root) -> src
sys.path.append(os.path.abspath(os.path.join(os.getcwd(), "../..../src")))

import Elements.pyECSS.math_utilities as util
from Elements.pyECSS.Entity import Entity
from Elements.pyECSS.Component import RenderMesh
from Elements.pyGLV.GL.Scene import Scene
from Elements.pyGLV.GL.Shader import InitGLShaderSystem, Shader,
↳ ShaderGLDecorator, RenderGLShaderSystem
from Elements.pyGLV.GL.VertexArray import VertexArray

```

```

def visualize_elements_scene(scene_objects_dict):
    """
    Visualizes the GANode objects using the Elements framework.
    """
    # 1. Initialize Scene
    scene = Scene()
    rootEntity = scene.world.createEntity(Entity(name="Root"))

    # 2. Camera Setup
    # Create a camera entity
    entityCam = scene.world.createEntity(Entity(name="Camera"))
    scene.world.addEntityChild(rootEntity, entityCam)

    # Camera parameters
    eye = util.vec(0.0, 5.0, 15.0)
    target = util.vec(0.0, 0.0, 0.0)
    up = util.vec(0.0, 1.0, 0.0)
    view = util.lookat(eye, target, up)
    # projMat = util.perspective(60.0, 1.33, 0.1, 100.0) # Perspective
    ↪projection
    projMat = util.ortho(-10.0, 10.0, -10.0, 10.0, 0.1, 100.0) # Orthographic
    ↪for better view of alignment

    # 3. Create Entities for each GANode
    # Define Cube Indices (Standard Triangulation)
    # Vertices are 8 points. We need 12 triangles (36 indices).
    # 0-3: Back, 4-7: Front
    indices = np.array([
        1,0,3, 1,3,2, # Back
        4,5,6, 4,6,7, # Front
        5,1,2, 5,2,6, # Right
        0,4,7, 0,7,3, # Left
        3,7,6, 3,6,2, # Top
        0,1,5, 0,5,4 # Bottom
    ], dtype=np.uint32)

    # Colors for different objects
    colors_palette = [
        [1.0, 0.0, 0.0, 1.0], # Red
        [0.0, 1.0, 0.0, 1.0], # Green
        [0.0, 0.0, 1.0, 1.0] # Blue
    ]

    for i, (name, ga_node) in enumerate(scene_objects_dict.items()):
        # Create Entity
        node = scene.world.createEntity(Entity(name=name))
        scene.world.addEntityChild(rootEntity, node)

```

```

# Create RenderMesh
mesh = scene.world.addComponent(node, RenderMesh(name=f"{name}_mesh"))

# Prepare Vertices
# GANode vertices are (N, 3). Elements expects (N, 4) homogeneous
↳ coords for some shaders,
# but standard COLOR_VERT_MVP usually takes vec4 position.
# Let's append w=1.0
verts_np = ga_node.vertices # (8, 3)
verts_homog = np.hstack((verts_np, np.ones((verts_np.shape[0], 1)))) #
↳ (8, 4)
verts_homog = verts_homog.astype(np.float32)

# Prepare Colors (Uniform color per object for simplicity)
color = colors_palette[i % len(colors_palette)]
colors_np = np.tile(color, (verts_np.shape[0], 1)).astype(np.float32)

# Attach attributes
mesh.vertex_attributes.append(verts_homog)
mesh.vertex_attributes.append(colors_np)
mesh.vertex_index.append(indices)

# Add VertexArray
vArray = scene.world.addComponent(node, VertexArray())

# Add Shader
# We use the basic COLOR shader with MVP matrix
shaderDec = scene.world.addComponent(node,
↳ ShaderGLDecorator(Shader(vertex_source=Shader.COLOR_VERT_MVP,
↳ fragment_source=Shader.COLOR_FRAG)))

# Calculate MVP
# Model matrix is Identity because vertices are already transformed in
↳ World Space by GA
model = util.identity()
mvpMat = projMat @ view @ model

shaderDec.setUniformVariable(key='modelViewProj', value=mvpMat,
↳ mat4=True)

# 4. Run Systems
print("Starting Elements Render Loop...")
print("Close the window to finish.")

# Initialize Window and Context

```

```

    scene.init(imgui=False, windowWidth=1024, windowHeight=768,
↪windowTitle="Elements: GA Scene Visualization")

    # Initialize Shaders
    initUpdate = scene.world.createSystem(InitGLShaderSystem())
    scene.world.traverse_visit(initUpdate, scene.world.root)

    # Render Loop
    renderUpdate = scene.world.createSystem(RenderGLShaderSystem())

    running = True
    while running:
        running = scene.render() # Handles events
        scene.world.traverse_visit(renderUpdate, scene.world.root)
        scene.render_post()

    scene.shutdown()

# Uncomment the line below to run the visualization
visualize_elements_scene(scene_objects)

```

UserWarning: Using SDL2 binaries from pysdl2-dll 2.32.0

Creating Scene Singleton Object

Creating ECSSManager Singleton Object

Starting Elements Render Loop...

Close the window to finish.

SDL2Window: init()

=====

Using OpenGL version 4.1

=====

OpenGL 4.1 Metal - 90.5 GLSL 4.10 Renderer Apple M3 Max

SDL2Window: shutdown()

2025-12-21 13:38:22.991 python[48415:3567291] TSM

AdjustCapsLockLEDForKeyTransitionHandling - _ISSetPhysicalKeyboardCapsLockLED

Inhibit

The Kernel crashed while executing code in the current cell or a previous cell.

Please review the code in the cell(s) to identify a possible cause of the
↪failure.

Click [here](https://aka.ms/vscodeJupyterKernelCrash) for more info.

View Jupyter [log](command:jupyter.viewOutput) for further details.